

# An Exact Solution to the Drop-the-Handkerchief Timing Game

The STL/DTH Project\*

July 30, 2026

## Abstract

Drop the Handkerchief (DTH) is a fully observed, two-player, zero-sum stochastic game with simultaneous exact-second actions, a hard 300-second capacity, and a probabilistic revival rule. We solve it. The state space is first collapsed by a per-player quotient: a player who can no longer survive an injection has a revival probability of identically zero, so their time-to-death coordinate is never read again and folds into a per-load sentinel, reducing the  $8.15 \times 10^9$ -state envelope to 289,374,121 equivalence classes, a factor of 28.2. A potential function that strictly increases along every live transition organizes those classes into 1,201 layers and licenses a single backward sweep with no search, no frontier, and no reachability analysis. Every class value is accepted only with an explicit equilibrium certificate: a mixed-strategy pair whose saddle gap against the fully reconstructed  $60 \times 60$  stage matrix is at most  $10^{-6}$ . The certificates come from a ladder of increasingly expensive certified solvers — an  $O(60)$  pure-saddle test, a pinned equalizer solve on guessed and on full supports, and linear programming — whose routing turned out to be dominated (99.8%) by full-support equilibria. The sweep was executed twice, by a Python reference implementation and by a parallel Rust kernel constrained to bit-identical arithmetic, and the two agree byte for byte on every value they both computed. The value of the complete game at the initial state is

$$V(0, 0, 0, 0) = 0.0898500728095116,$$

so under optimal play the first Dropper wins with probability  $\approx 0.5449$ . The entire computation took about 1.7 hours of wall clock on one twelve-core consumer desktop, against a previously recorded single-core projection of 3.7–5.1 years.

## 1 Introduction

DTH is the pure timing core of the “Surpassing the Leader” wager from the manga *Usogui*; the rules modeled here are exactly those licensed by the project’s evidence ledger, and the game sits at rung L1 of the repository’s formulation ladder (`docs/FORMULATION_LADDER.md`): literal one-second actions, no leap-second window, no private information. Two players alternate the roles of *Dropper* and *Checker*. Each round both simultaneously commit to a second in  $\{1, \dots, 60\}$ ; the check succeeds if it does not precede the drop, in which case the checker absorbs the inclusive elapsed interval into a personal 300-second cylinder, and the roles swap. A failed check injects the checker with their

---

\*Maintained at `paper/dth_exact_solution.tex` in the project repository. Every constant reported here is reproducible from the repository head: the solver is `src/dth/backup_tablebase.py` with the kernel `src/crates/dth_backup`, the artifact schema and cross-implementation contract is `src/dth/docs/DTH_BACKUP_PARITY.md`, and the rules evidence ledger is `docs/papers/game-sources/EVIDENCE.md`.

accumulated load plus a fixed 60-second penalty; the injection is fatal unless a frozen, monotone revival law saves them, in which case their cylinder resets and their cumulative time dead grows. Overflowing the cylinder is fatal outright. The last player alive wins.

The game is finite but large, stochastic, and genuinely simultaneous: its stage games require mixed strategies, and its state space was, until now, handled by bounded search around certified anchors. This paper records the complete solution and the small amount of mathematics that made it cheap:

1. a *per-player quotient* (Section 4): death eligibility is absorbing and zeroes the only expression that ever reads a player’s time-to-death, so that coordinate collapses to a sentinel independently for each player;
2. a *potential* (Section 5) that strictly increases along every live transition, so a single descending sweep sees every child before its parents — eliminating search, reachability analysis, and frontier bookkeeping in one stroke;
3. a *certificate discipline* (Section 6): no value enters the table without a mixed-strategy pair whose one-step exploitability against the fully reconstructed stage matrix is at most  $\tau_0 = 10^{-6}$ , no matter which solver produced it — so the internals of the fast solvers are irrelevant to soundness; and
4. the 61-class *Toeplitz structure* of the stage matrices (Section 3), which makes certificates and pure-saddle tests  $O(60)$  or  $O(60k)$  instead of  $O(60^2)$ .

Section 7 reports the computation itself, its cross-implementation audit, and the error analysis; Section 8 states precisely what is and is not established.

## 2 The game

**Definition 2.1** (State). *A live state is  $x = (s_c, t_c, s_d, t_d) \in \mathcal{X}$ , written from the perspective of the current Dropper:  $s_c \in \{0, \dots, 299\}$  and  $t_c \in \{0, \dots, 300\}$  are the current Checker’s accumulated load (“squandered time”, *ST*) and cumulative time dead (*TTD*), and  $(s_d, t_d)$  are the Dropper’s. Terminal outcomes are **W** (the mover wins, utility +1) and its negation under the role swap.*

**Definition 2.2** (Actions and timing). *Both players simultaneously choose seconds  $d, c \in \{1, \dots, 60\}$  (drop and check). The check succeeds iff  $d \leq c$ , and then the checker’s load grows by the inclusive interval  $\ell = c - d + 1 \in \{1, \dots, 60\}$ ; equal seconds cost one second, not zero. If  $s_c + \ell \geq 300$  the cylinder overflows and the checker dies: the transition is terminal **W** for the Dropper. Otherwise play continues from*

$$(s_c, t_c, s_d, t_d) \xrightarrow{\text{success, lag } \ell} (s_d, t_d, s_c + \ell, t_c),$$

*with the roles swapped (the old Checker drops next).*

**Definition 2.3** (Failure, dose, and revival). *On a failed check ( $d > c$ ) the checker is injected with the dose  $q = s_c + 60$ . Survival is possible iff*

$$q < 300 \ \wedge \ q + t_c \leq 300 \quad \Longleftrightarrow \quad s_c \leq 239 \ \wedge \ s_c + t_c \leq 240, \tag{1}$$

*where the second inequality is deliberately non-strict: a resulting cumulative *TTD* of exactly 300 remains eligible. When (1) holds, the checker revives with the frozen probability*

$$p(s_c, t_c) = 0.95 \left(1 - \frac{s_c}{240}\right) 0.75^{t_c/60}, \tag{2}$$

their cylinder resets, and their TTD grows by the dose:

$$(s_c, t_c, s_d, t_d) \xrightarrow{\text{failure, revive}} (s_d, t_d, 0, t_c + s_c + 60) \quad \text{with probability } p(s_c, t_c),$$

else the transition is terminal  $W$ . When (1) fails, the injection is fatal with certainty and  $p \equiv 0$ .

Values are always from the current Dropper's perspective and negate across the role swap; the game is zero-sum with terminal-only payoff in  $\{-1, +1\}$ , so all values lie in  $[-1, 1]$ . We call the checker profile  $(s, t)$  *dead* when (1) fails — the player is still playing, but any future failed check is certainly fatal for them.

### 3 Finiteness and stage structure

**Lemma 3.1** (Damage strictly increases). *Let  $R(x) = s_c + t_c + s_d + t_d$ . Every live transition strictly increases  $R$ : a successful lag  $\ell$  adds  $\ell \geq 1$ , and a revived failure adds exactly 60 (the dose  $s_c + 60$  enters TTD while the load  $s_c$  resets).*

**Corollary 3.2** (The game is finite).  *$R \leq 299 + 300 + 299 + 300 = 1198$  on live states, so every play reaches a terminal outcome within 1198 live transitions. The game therefore has a unique value  $V : \mathcal{X} \rightarrow [-1, 1]$ , obtained by backward induction over the finite acyclic transition graph [1], solving at each state the zero-sum simultaneous stage game by von Neumann's minimax theorem [2]; the construction is the finite, absorbing case of a Shapley stochastic game [3].*

**Proposition 3.3** (The 61 transition classes). *Fix a live state  $x$  and let  $V$  assign values to its children. Define*

$$S_\ell = \begin{cases} +1, & s_c + \ell \geq 300, \\ -V(s_d, t_d, s_c + \ell, t_c), & \text{else,} \end{cases} \quad F = \begin{cases} +1, & (s_c, t_c) \text{ dead,} \\ p(-V(s_d, t_d, 0, t_c + s_c + 60)) + (1 - p), & \text{else.} \end{cases}$$

Then the  $60 \times 60$  stage matrix of  $x$ , with the Dropper as row player, is

$$M(x)_{d,c} = \begin{cases} S_{c-d+1}, & c \geq d, \\ F, & c < d, \end{cases} \quad (3)$$

and  $V(x) = \text{val}(M(x))$ . In particular  $M(x)$  carries only 61 distinct entries: it is Toeplitz on and above the diagonal and constant below it.

*Proof.* A successful cell depends on the actions only through the lag  $\ell = c - d + 1$ , and every failed cell has the identical transition distribution; the entries are the expected continuation values, negated for the role swap. The Bellman equation is Corollary 3.2.  $\square$

**Lemma 3.4** ( $O(60)$  saddle reductions). *Row  $d$  of (3) contains exactly the prefix  $S_1, \dots, S_{61-d}$  together with  $F$  when  $d > 1$ , and column  $c$  contains the prefix  $S_1, \dots, S_c$  together with  $F$  when  $c < 60$ . Hence with prefix scans  $m_j = \min_{i \leq j} S_i$  and  $M_j = \max_{i \leq j} S_i$ ,*

$$\max_d \min_c M(x)_{d,c} = \max_d \begin{cases} m_{60}, & d = 1, \\ \min(m_{61-d}, F), & d > 1, \end{cases} \quad \min_c \max_d M(x)_{d,c} = \min_c \begin{cases} \max(M_c, F), & c < 60, \\ M_{60}, & c = 60, \end{cases}$$

computable in  $O(60)$  operations, exactly (min and max are associative and commutative on floats), from the 61 class values alone.

## 4 The per-player dead-TTD quotient

The complete game reads a player’s TTD in exactly one expression: the revival probability (2) of a failed check. That single fact powers the entire reduction.

**Lemma 4.1** (Death is absorbing). *If the profile  $(s, t)$  is dead, then  $(s', t)$  is dead for every  $s' \geq s$ . Since a successful check raises a player’s load and leaves their TTD unchanged, and a failed check either ends the game for them or requires (1), load growth is the only motion available to a dead profile, and it never revives it.*

**Theorem 4.2** (Quotient soundness). *Define  $\varphi(s, t) = (s, t)$  when  $(s, t)$  is alive and  $\varphi(s, t) = (s, \text{dead})$  otherwise, applied to each player’s profile independently. Then any two live states with equal images under  $\varphi$  have identical stage matrices up to  $\varphi$ -identification of their children, and hence identical values:  $V$  factors through  $\varphi$ .*

*Proof.* Compare the 61 classes of Proposition 3.3 at two states that differ only in a dead player’s TTD. Success classes never read TTD and preserve both profiles’ aliveness classification by Lemma 4.1, so they map to  $\varphi$ -identified children. The failure class reads the checker’s TTD only through  $p$ : if the checker profile is dead,  $p \equiv 0$  and  $F = +1$  with no child at all; if alive, the TTD is not quotiented and  $F$  is read off unchanged. Induction over the finite acyclic graph (Corollary 3.2) closes the argument.  $\square$

**Proposition 4.3** (Counting). *Over the transition-closed TTD domain  $\{0\} \cup [60, 300]$  (Lemma 4.4), a player has  $300 \times 242 = 72,600$  profiles;  $\varphi$  collapses them to 16,711 alive profiles plus 300 dead sentinels (one per load), i.e. 17,011 classes per player and*

$$17,011^2 = 289,374,121$$

*two-player classes — a factor 28.2 below the raw  $300 \cdot 301 \cdot 300 \cdot 301 = 8,154,090,000$  state envelope.*

**Lemma 4.4** (Domain closure). *Every root of interest has both TTDs in  $\{0\} \cup [60, 300]$ , and the domain is transition closed: revival requires  $s_c + t_c \leq 240$ , so a failure child’s TTD  $t_c + s_c + 60$  lies in  $[60, 300]$ , and success preserves TTDs. A live state with an alive TTD in 1..59 is thus unreachable from the domain; the implementation rejects such states rather than approximate them. A dead TTD is discarded by  $\varphi$  exactly, so any value is accepted there.*

All four collapse claims — the closed form (1), the vanishing of (2) on dead profiles, absorption, and the counts — are locked by exhaustive machine enumeration over all 72,600 profiles (`src/dth/tests/test_dead_ttd_quotient.py`, `test_packed_class_codec.py`).

## 5 The potential and the sweep schedule

**Definition 5.1** (Potential). *For a profile let  $\rho(s, t) = t$  if  $(s, t)$  is alive and  $\rho(s, t) = 301$  otherwise; for a class let*

$$\Phi(x) = s_c + s_d + \rho(s_c, t_c) + \rho(s_d, t_d) \in \{0, \dots, 1200\}.$$

**Theorem 5.2** (Strict increase). *Every live transition strictly increases  $\Phi$ . By cases on the moving profile:*

- *successful lag  $\ell$ , profile stays alive or stays dead:  $\Delta\Phi = \ell \geq 1$ ;*

- *successful lag  $\ell$ , alive profile dies:  $\Delta\Phi = \ell + 301 - t \geq \ell + 61$ , since alive implies  $t \leq 240$  (and a dead profile never revives, so the reverse case is empty);*
- *failure, revived profile alive:  $\Delta\Phi = 60$  (the load resets to zero while TTD absorbs the dose);*
- *failure, revived profile dead:  $\Delta\Phi = 301 - (s_c + t_c) \geq 61$  by (1).*

**Corollary 5.3** (Layered backward induction). *No transition stays within a  $\Phi$ -layer, so solving whole layers in descending  $\Phi$  sees every child before any parent, and layer- $P$  residual work need only complete before layer  $P-1$  begins. The 289,374,121 classes occupy all 1,201 layers; the largest,  $\Phi = 374$ , holds 1,678,715 classes. A layer is a union of bucket rectangles: grouping profiles by their own potential  $s + \rho \in [0, 600]$ , layer  $P$  is exactly  $\bigcup_a \text{bucket}(a) \times \text{bucket}(P - a)$ .*

The strict increase is verified exhaustively over all 1,035,541 live profile transitions (`src/dth/tests/test_backup_potential.py`); the schedule needs no reachability analysis, no frontier queue, and no transposition table — unreachable classes are simply solved too.

## 6 Certified equilibria

**Definition 6.1** (Certificate). *For a stage matrix  $M$  and mixed strategies  $\sigma$  (Dropper) and  $\tau$  (Checker), let*

$$L = \min_c (\sigma^\top M)_c, \quad U = \max_d (M\tau)_d, \quad \text{gap}_M(\sigma, \tau) = \max(0, U - L).$$

*The pair certifies the value  $\hat{v} = (L + U)/2$  at tolerance  $\tau_0$  when  $\text{gap}_M(\sigma, \tau) \leq \tau_0$ . Throughout,  $\tau_0 = 10^{-6}$ , and the gap is always measured against the full matrix (3), never a submatrix.*

**Proposition 6.2** (Certified midpoint).  *$L \leq \text{val}(M) \leq U$ , so a certificate at tolerance  $\tau_0$  pins  $|\hat{v} - \text{val}(M)| \leq \tau_0/2$ .*

*Proof.*  $\sigma$  guarantees the row player at least  $L$  against every reply, and  $\tau$  caps the row player at  $U$ ;  $\text{val}(M)$  sits between the two security levels.  $\square$

Two classical stability facts transfer certificate quality across perturbed inputs; both are retained from the project’s earlier whitepaper and cited elsewhere in the repository by these names.

**Lemma 6.3** (Matrix-value Lipschitz bound). *If  $\|M - \widehat{M}\|_\infty \leq \epsilon$  entrywise, then  $|\text{val}(M) - \text{val}(\widehat{M})| \leq \epsilon$ , since  $|\sigma^\top (M - \widehat{M}) \tau| \leq \epsilon$  for all mixed pairs.*

**Proposition 6.4** (Approximate-matrix saddle gap). *If  $(\hat{\sigma}, \hat{\tau})$  is an exact equilibrium of  $\widehat{M}$  and  $\|M - \widehat{M}\|_\infty \leq \epsilon$ , then  $\text{gap}_M(\hat{\sigma}, \hat{\tau}) \leq 2\epsilon$ : play computed from an  $\epsilon$ -accurate matrix is  $2\epsilon$ -unexploitable in the true one.*

### 6.1 The solver ladder

Each class is routed through a fixed ladder of certified solvers; the certificate, not the solver, is the acceptance test, so a rung may fail closed for any reason without affecting soundness.

1. **Pure saddle.** Lemma 3.4 computes the maximin and minimax of pure strategies in  $O(60)$ ; if they meet within  $\tau_0$  the midpoint is certified by the corresponding pure pair.

2. **Equalizer supports.** For a guessed support pair  $(R, C)$  with  $k = \min(|R|, |C|)$ , solve the two  $(k+1) \times (k+1)$  equalizer systems

$$\begin{pmatrix} M_{R,C} & -\mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix} \begin{pmatrix} \tau_C \\ w \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ 1 \end{pmatrix}, \quad \begin{pmatrix} M_{R,C}^\top & -\mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix} \begin{pmatrix} \sigma_R \\ v \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ 1 \end{pmatrix},$$

reject on singularity or negative mass, and certify the lifted pair against the full matrix in  $O(60k)$  via (3). Guessed supports come from spatial neighbours one potential step up (the lag-one shift of either profile), recorded when the neighbour was solved. The case  $k = 60$  is the classical full-support structured solve and needs no guess.

3. **Linear programming.** The standard primal LP for the row player with the column strategy extracted from its inequality duals, falling back to two independent LPs, and finally to the same two LPs under tightened solver feasibility tolerances ( $10^{-10}$ , dual simplex). The retry tightens the *solver*, never the gate: every answer is certified at  $\tau_0$  against the full matrix, and a matrix failing all rungs aborts the computation rather than store an uncertified value.

## 6.2 Propagated error

The table stores certified midpoints, so a parent’s matrix is assembled from child values that are themselves within a bounded distance of truth.

**Proposition 6.5** (A-priori accumulated bound). *Let every stored value carry a certificate at tolerance  $\tau_0$  and let  $\delta(x)$  bound the longest live path from  $x$  to a terminal. Then the stored value  $\hat{V}(x)$  satisfies*

$$|\hat{V}(x) - V(x)| \leq \delta(x) \frac{\tau_0}{2} \leq \frac{1200 - \Phi(x)}{2} \tau_0.$$

*In particular the initial state obeys  $|\hat{V} - V|(0, 0, 0, 0) \leq 6.0 \times 10^{-4}$ .*

*Proof.* Induction up the DAG. Terminal entries are exact. If every child satisfies the bound with  $\delta(x) - 1$ , the assembled matrix is entrywise within  $(\delta(x) - 1)\tau_0/2$  of the true one (expectations and negation are non-expansive), Lemma 6.3 transfers that to the value, and Proposition 6.2 adds the state’s own  $\tau_0/2$ . Each live transition raises  $\Phi$  by at least one (Theorem 5.2), bounding  $\delta$ .  $\square$

This a-priori bound is loose by design; Section 7 reports the measured gaps, which are typically at the  $10^{-12}$  scale and never exceeded  $4.9 \times 10^{-7}$  on any audited sample, and every stored value’s certificate is re-derivable on demand from the stored children (Bellman recertification), so no per-class certificate needs to be persisted.

## 7 The computation

### 7.1 Artifact and schedule

Classes are addressed as  $p_c \cdot 17,011 + p_d$  over the normative profile enumeration (TTD-ascending, load-ascending, dead sentinels by load); the artifact is one dense 2.16 GiB float64 array plus one routing byte per class, memory mapped, with per-layer atomic checkpoints and byte-for-byte resume. All revival probabilities (2) are evaluated once, in one place, and every downstream kernel only gathers from those tables — the kernels perform no transcendental arithmetic.

## 7.2 Cross-implementation audit

The sweep is implemented twice: a Python reference that defines the semantics, and a parallel Rust kernel constrained to *bit-identical* arithmetic — pinned elimination order, first-maximum pivoting, no fused multiply-add, sequential ascending reductions, and per-state warm-start guesses precomputed so that no kernel result depends on thread count or work partition. On every region computed by both (all synthetic end-to-end sweeps, and a forked continuation of the production artifact) the two implementations produced byte-identical value and routing arrays; the contract and its test suite are `src/dth/docs/DTH_BACKUP_PARITY.md`. The LP residue rung is shared Python code by construction. This does not make the two implementations independent in the cryptographic sense, but it does mean every accepted value passed the certificate check in two codebases in two languages, and that the parallel kernel cannot have introduced scheduling nondeterminism.

## 7.3 Results

The full sweep over all 289,374,121 classes completed in roughly 1.7 hours of wall clock across checkpointed sessions on a single twelve-core consumer desktop (AMD Ryzen 9 3900X), against the previously recorded single-core projection of 3.7–5.1 years for the unquotiented, search-based design. Solver routing:

| rung                                 | classes     | share   |
|--------------------------------------|-------------|---------|
| pure saddle (Lemma 3.4)              | 334,588     | 0.116%  |
| equalizer, warm-started support      | 38,138      | 0.013%  |
| equalizer, full support ( $k = 60$ ) | 288,810,400 | 99.805% |
| linear programming                   | 190,995     | 0.066%  |
| of which dual-extraction single LP   | 190,986     |         |
| of which tightened-tolerance retry   | 9           |         |

Complete-game DTH equilibria are thus overwhelmingly full-support — a sharp contrast with the small-support statistics previously measured on shallow finite-horizon matrices, and the reason the classical structured solve, not support enumeration, is the workhorse.

Audits on the finished artifact: a completeness scan (every value finite, in  $[-1, 1]$ , with a known routing byte); Bellman recertification of 4,792 deterministically sampled classes with worst deviation  $4.84 \times 10^{-7}$ ; agreement with the independently produced SQLite band artifact (`exact_band_v1`, both-loads  $\geq 240$  quotient) on all 3,541 shared classes to at most  $4.9 \times 10^{-8}$ , and at its root to  $6.9 \times 10^{-12}$ ; and an independent dead-band reference solver sharing no sweep machinery. Selected certified values (recertification deviation in parentheses):

| state $(s_c, t_c, s_d, t_d)$ | $\hat{V}$          |                         |
|------------------------------|--------------------|-------------------------|
| (0, 0, 0, 0) initial         | 0.0898500728095116 | $(1.4 \times 10^{-15})$ |
| (240, 0, 240, 0) band root   | 0.3372132166359819 | $(3.3 \times 10^{-16})$ |
| (239, 0, 0, 240) anchor      | 0.9946124326578454 | $(4.4 \times 10^{-16})$ |

**Theorem 7.1** (Main result). *The complete Drop-the-Handkerchief game of Section 2, restricted to the transition-closed quotient domain of Lemma 4.4, is solved: every one of the 289,374,121 classes carries a stored value certified at saddle-gap tolerance  $10^{-6}$ , with the a-priori accumulated bound of Proposition 6.5 and the measured audits above. At the initial state,*

$$\hat{V}(0, 0, 0, 0) = 0.0898500728095116,$$



so under optimal play the first Dropper wins with probability  $\frac{1+\hat{V}}{2} \approx 0.5449$ , within the stated error bounds. Equilibrium strategies at any class are reconstructible on demand from the stored child values through the same certified ladder.

## 8 Scope of the claims

- **Rules-relative.** The theorems are about the game defined in Section 2, i.e. formulation rung L1 with the frozen revival surface (2). Nothing here solves the leap-aware “Surpassing the Leader” game (rung L2 and above), which adds an asymmetric 61st second and is a different game.
- **Certified numerics, not rational arithmetic.** Values are IEEE double midpoints with explicit saddle-gap certificates and the accumulated bound of Proposition 6.5; this is a certified-numerics result, not an exact rational or interval-arithmetic proof. The structural theorems (quotient, potential, finiteness, Toeplitz form) are exact and machine-checked exhaustively.
- **Domain.** The table covers the transition-closed quotient domain; live states with an alive TTD in 1..59 are outside it and are rejected, not approximated (Lemma 4.4).
- **Values, not tabulated policies.** The artifact stores values and routing only; policies are re-derived (and re-certified) per query.

## Reproducibility

From the repository head: `uv run python -m dth backup` builds the artifact (checkpointed, resumable; the configuration is `src/dth/config/backup_full_v1.yaml`); the exhaustive structural tests and both-backend parity suite run under `uv run python -m pytest src/dth/tests -q` and `cargo test --workspace`. The artifact manifest embeds SHA-256 digests of every array and of the solver sources that produced it.

## References

- [1] Ernst Zermelo. Über eine Anwendung der Mengenlehre auf die Theorie des Schachspiels. In *Proceedings of the Fifth International Congress of Mathematicians*, volume 2, pages 501–504, Cambridge, 1913.
- [2] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100:295–320, 1928. <https://doi.org/10.1007/BF01448847>
- [3] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953. <https://doi.org/10.1073/pnas.39.10.1095>